MANAR: Supervised Autonomy, Deterministic Control, and Greedy Search Route Optimization for Aerial Search-and-Rescue

Oumar Ibrahim
MANAR Search-and-Rescue Project
United Arab Emirates

***Abstract*—Aerial Search-and-Rescue (SAR) operations require systematic area coverage, dependable navigation, and robust operational safety considerations. This paper presents the architecture, mathematical design, and functional prototype of MANAR (Multi-sensor Autonomous Navigation and Aerial Rescue), a supervised-autonomy SAR system. We draw a clear distinction across three system maturity tiers: the currently implemented C++ functional prototype, the selected closed software architecture, and planned future perception extensions. The core technical contribution focuses on MANAR's deterministic control flow, Haversine geographic navigation, and an $O(n^2)$ greedy nearest-neighbor search route optimizer. Crucially, the system enforces operator override privilege: automated route optimization assists the operator without overriding manually entered sector sequences when tactical domain knowledge dictates search ordering. We present parameterized mathematical models for mode-dependent velocity profiles, simulated battery failsafe progression, and systematic lawnmower grid search. Functional prototype verification confirms deterministic control behavior, search-plan locking, and Return-to-Launch (RTL) navigation preemption.**

***Index Terms*—Search-and-Rescue UAVs, Deterministic Control, Supervised Autonomy, Greedy Route Optimization, Haversine Navigation, Autonomous Systems.**

I. Introduction

Search-and-Rescue (SAR) missions represent a critical operational domain for unmanned aerial systems [1]. When disoriented personnel, disaster survivors, or missing individuals are stranded in vast or remote environments, emergency response windows are severely constrained by environmental conditions and terrain access. Unmanned Aerial Vehicles (UAVs) provide rapid deployment capabilities, flexible sensor mobility, and elevated aerial coverage.

MANAR (Multi-sensor Autonomous Navigation and Aerial Rescue) is a supervised-autonomy aerial search-and-rescue system designed to balance human oversight against automated execution [2]. The term *Autonomous* in MANAR refers to autonomous navigation capabilities within a supervised-autonomy system, maintaining explicit human supervisory authority over high-risk mission actions. Fully manual teleoperation imposes heavy cognitive workloads on operators, while black-box autonomous systems introduce risks of uncoordinated behavior during sensor dropouts or communication loss. In MANAR, human operators maintain supervisory authority over mission configuration and target verification, while a C++ control core executes deterministic navigation, route optimization, and safety monitoring.

TABLE I
MANAR Capability Implementation Status

Capability	Status
Deterministic C++ control core	✓ Implemented
Haversine navigation geometry	✓ Implemented
Multi-location sector sequencing	✓ Implemented
Greedy route optimization ($O(n^2)$)	✓ Implemented
Operator optimization toggle	✓ Implemented
Lawnmower search grid logic	✓ Implemented
Payload component state controls	✓ Implemented
Structured event logging	✓ Implemented
React / TypeScript Web GUI	Selected
WebSocket + JSON IPC	Selected
Centralized state persistence	Refactoring
Automated perception pipeline	Planned
Multi-sensor evidence fusion	Planned
PX4 SITL / HITL simulation	Planned
Physical flight deployment	Planned

This paper categorizes MANAR's design across three distinct system maturity tiers:

1) **Implemented Functional Prototype:** C++ application logic (`control.exe`, `terminal.exe`), Haversine geographic distance calculations, $O(n^2)$ greedy nearest-neighbor route optimization with operator override, search plan locking, sequential lawnmower search progression, discrete battery simulation rules, and timestamped structured plain-text event logging.
2) **Selected Closed Architecture:** Centralized authoritative state ownership by the C++ control layer, React/TypeScript operator interface communicating via WebSocket + JSON, and on-demand Return-to-Launch (RTL) navigation semantics.
3) **Planned Future Work:** Automated Python/PyTorch perception models, multi-sensor evidence fusion algorithms, physical SITL/HITL flight controller integration, and field deployment testing.

Table I summarizes the implementation maturity of MANAR's core capabilities.

A. Key Contributions

The main contributions of this work are:

- Formulation and implementation of an $O(n^2)$ greedy nearest-neighbor route optimizer using Haversine geodetics [3], explicitly incorporating an operator override mechanism to preserve human domain knowledge.
- Formalization of parameterized motion and simulated battery failsafe progression models governing deterministic mission execution.
- Specification of MANAR's selected software architecture separating C++ control logic, web-based operator interfaces, and future perception pipelines.
- Functional verification of search plan locking, sequential sector progression, and Return-to-Launch navigation preemption.

II. System Requirements

MANAR is engineered to fulfill operational constraints tailored for aerial search-and-rescue. This section documents the operational envelope, symbolic flight parameters, search geometry, and safety monitoring rules.

A. Operational Velocity Presets

The flight system defines four discrete velocity presets ($v(m)$) suited for distinct mission phases:

- **Quick Mode (v_Q):** High-speed transit between distant search sectors or rapid return to base.
- **Active Mode (v_A):** Baseline search velocity during systematic lawnmower sector sweeps.
- **Inspect Mode (v_I):** Low-speed visual inspection of candidate detection sites.
- **Hover Mode (v_H):** Stationary hold over candidate locations ($v_H = 0$).

Velocity presets are configurable parameters governed by configuration profiles. The software enforces a configurable altitude ceiling clamp $H_{\max}$ and maximum velocity clamp $v_{\max}$. Default takeoff climb target is specified by H_{launch}.

B. Operator Supervision Principles

In accordance with supervised autonomy design:

- The system retains human supervisory authority over high-risk mission actions and physical state changes.
- The human operator maintains real-time override authority to trigger Return-to-Launch (RTL) or issue manual waypoint updates.
- The operator retains explicit authority over whether automated search route optimization is applied to entered locations.

C. Search Grid Parameters

Systematic sector coverage requires structured grid patterns. Search sectors are parameterized by:

- Lawnmower search sector dimensions: Width W and Height H.
- Parallel sweep row spacing: Δy.
- Waypoint arrival acceptance radius: R_{reach}.
- Home base arrival acceptance radius: R_{home}.

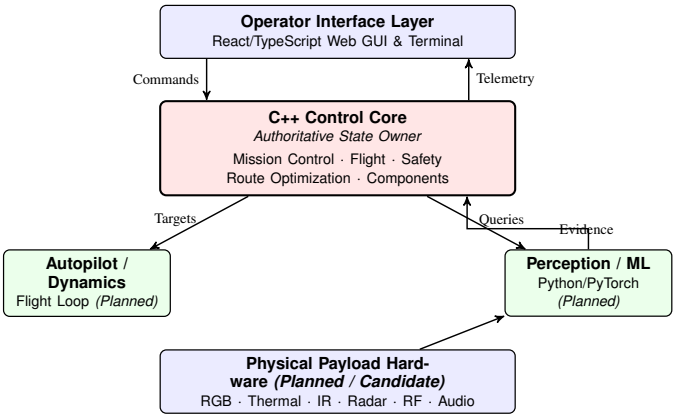

Fig. 1. MANAR system architecture. The C++ Control Core is the selected authoritative state owner.

D. Battery Failsafe Hierarchy

To exercise deterministic safety monitoring, the system defines four configurable battery percentage thresholds ($B_{\text{warn}}, B_{\text{rtl\_warn}}, B_{\text{emergency\_rtl}}, B_{\text{land}}$):

- **Battery Warning ($B \leq B_{\text{warn}}$):** Logs a warning and activates configured battery-save mode.
- **Recommend RTL ($B \leq B_{\text{rtl\_warn}}$):** Recommends Return-to-Launch navigation and transmits aircraft status and location.
- **Emergency RTL ($B \leq B_{\text{emergency\_rtl}}$):** Automatically overrides active search patterns and commands Return-to-Launch navigation via `activateRTL()`.
- **Emergency Land ($B \leq B_{\text{land}}$):** Invokes the prototype stop-flight path, records the current position, and transitions simulated flight state to stopped.

III. System Architecture

MANAR's architecture establishes clear functional boundaries across operational layers. Fig. 1 illustrates core components and data flows.

A. Selected State Ownership Architecture

MANAR's selected final architecture establishes that the C++ Control Core owns authoritative runtime state. Under this design, state modifications requested by operators or triggered by safety rules are evaluated and committed exclusively by the C++ control layer (`control.exe`). Subsystem modules update internal state variables and report telemetry to the core rather than directly mutating shared state structures.

Implementation Status Note: In the current software prototype, several subsystem functions still persist runtime data directly to disk via `saveRuntime()`. These calls are being systematically refactored toward the centralized state-ownership model.

B. Communication & IPC Architecture

The communication architecture distinguishes between current prototype IPC and the selected final interface design:

- **Current Prototype Data Flow:** File-backed JSON contracts under `runtime/` handle prototype communication and state persistence: `commands.json` serves as the command/request transport from the terminal to the control application, `runtime.json` provides the live prototype status representation, and `search_locations.json` stores the persisted locked search-plan representation written by `control.cpp` upon processing location entry commands.
- **Selected Final Architecture:** Real-time WebSocket + JSON protocol between the React/TypeScript GUI and C++ control core. In this target model, `runtime.json` is retained as an on-demand snapshot export rather than the live transport.

IV. Prototype Mission Control Flow

MANAR manages mission progression through deterministic control flow logic implemented in `mission.cpp`.

A. State Representation & Flag Logic

Rather than a rigid finite state machine, the current C++ prototype tracks mission lifecycle transitions through explicit state flags:

- `missionstarted` (`bool`): Indicates whether a mission is active.
- `searchplanlocked` (`bool`): Indicates whether search location coordinates are frozen.
- `enroute` (`bool`): Indicates active transit toward a target search sector.
- `destinationsearched` (`bool`): Set when lawnmower sweep over the active sector completes.
- `rescueefound` (`bool`): Flag representing target detection.
- `waitingforhelp` (`bool`): Enforces a stationary hold upon candidate detection.
- `isdestinationhome` (`bool`): Set when the active navigation target is the home base station.

B. Sequential Sector Progression

Multi-location search plans are executed sequentially. Each entry in `search_locations.json` represents a search sector origin $(L_1, L_2, \ldots, L_N)$.

When lawnmower sweeps over location k ($1 \leq k \leq N$) finish, `lawnmower()` sets `destinationsearched = true`. On the next loop iteration of `missionstatusupdater()`, the control core advances the index:

$$k \leftarrow k + 1 \tag{1}$$

resets lawnmower sweep variables (`searchrow = 0`, `horizontalmove = true`, `moveeast = true`), loads location $k + 1$ as the new destination, and updates `runtime["search"]["current_location_id"] = k + 1`. Once all N sectors have been searched ($k = N$), the system initiates Return-to-Launch navigation.

C. Candidate Detection and Rescuee Representation

In the current prototype, candidate victim detection is exercised using a manual `rescueefound` trigger flag. When set, `configurerescueestate()` executes `stopflight()`, halting horizontal movement, switching flight mode to `Stall`, turning off active payload components to conserve battery, and setting `waitingforhelp = true`.

Automated perception algorithms will replace this manual trigger flag when the ML detection pipeline is implemented.

D. Return-to-Launch (RTL) Semantics

MANAR's selected architecture defines Return-to-Launch (RTL) as an on-demand navigation command that sets or changes the target destination to the configured launch location. It is not conceptually a permanent returning mission mode.

Current Prototype Behavior: In the current software implementation, `activateRTL()` handles RTL preemption by checking if the aircraft is landed (calling `launch()` and engaging configured active flight mode if grounded), clearing rescue flags (`rescueefound = false`, `waitingforhelp = false`), overwriting `search_locations.json` with a single home destination entry (`planning_mode = "rtl_home"`), and locking `isdestinationhome = true`.

V. Mathematical Model

MANAR incorporates geographic navigation geometry, flight dynamics rules, and battery simulation models to govern aircraft behavior.

A. Geographic Distance (Haversine Formulation)

To evaluate spatial relationships over Earth's spherical surface without relying on planar approximations, MANAR calculates geographic distance using the Haversine formula.

Let two points on the Earth's surface be defined by their geodetic coordinates:

$$P_1 = (\phi_1, \lambda_1), \qquad P_2 = (\phi_2, \lambda_2) \tag{2}$$

where ϕ represents latitude and λ represents longitude in radians.

The angular differences are:

$$\Delta\phi = \phi_2 - \phi_1, \qquad \Delta\lambda = \lambda_2 - \lambda_1 \tag{3}$$

The square of half the chord length between the points is given by:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{4}$$

The angular distance in radians is:

$$c = 2\,\mathrm{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \tag{5}$$

The surface distance $d(P_1, P_2)$ in meters is:

$$d(P_1, P_2) = R_E \cdot c \tag{6}$$

where $R_E = 6\,371\,000.0$ m is mean Earth radius. In `route_optimizer.cpp` and `drone.cpp`, distance calculations use this exact double-precision Haversine formulation.

B. Motion Model and Estimated Time of Arrival (ETA)

The aircraft's horizontal velocity $v(m)$ is determined by the active operational preset $m \in \{\text{Quick}, \text{Active}, \text{Inspect}, \text{Hover}\}$:

$$v(m) = \begin{cases} v_Q & m = \text{Quick} \\ v_A & m = \text{Active} \\ v_I & m = \text{Inspect} \\ v_H & m = \text{Hover} \end{cases} \quad \text{(m/s)} \tag{7}$$

where $v_H = 0$ is defined semantically as stationary hover.

Given current aircraft coordinates P_{drone} and target coordinates P_{dest}, remaining distance is $d = d(P_{\text{drone}}, P_{\text{dest}})$. For $v(m) > 0$, Estimated Time of Arrival (t_{ETA}) is formulated as:

$$t_{\text{ETA}} = \frac{d(P_{\text{drone}}, P_{\text{dest}})}{v(m)} \quad \text{(seconds)} \tag{8}$$

C. Discrete Battery Simulation Model

To exercise battery monitoring and failsafe behavior in the software prototype without requiring physical voltage/current telemetry, battery percentage $B_k \in [0, 100]$ evolves according to a discrete simulation rule:

$$B_{k+1} = \max\left(0.0,\ B_k - \alpha_m \cdot \Delta B\right) \tag{9}$$

where $\Delta B > 0$ is a step decrement constant and α_m is a mode-dependent prototype simulation coefficient. When the aircraft is landed ($v = 0$, $H = 0$), battery consumption is paused ($\Delta B = 0$). This discrete simulation rule exists solely to exercise software battery and failsafe behavior, and is not a physical electro-aerodynamic battery model.

D. Lawnmower Search Grid Geometry

For a sector centered at (ϕ_c, λ_c) with sector width W along longitude and height H along latitude, conversion between meters and geographic coordinates uses local scale factors:

$$M_{\text{lat}} = 111\,320.0 \quad \text{(m / deg lat)} \tag{10}$$

$$M_{\text{lon}}(\phi_c) = 111\,320.0 \cdot \cos\left(\frac{\phi_c \cdot \pi}{180.0}\right) \quad \text{(m / deg lon)} \tag{11}$$

The southwest origin corner (ϕ_0, λ_0) of the grid is computed as:

$$\phi_0 = \phi_c - \frac{H/2.0}{M_{\text{lat}}}, \qquad \lambda_0 = \lambda_c - \frac{W/2.0}{M_{\text{lon}}(\phi_c)} \tag{12}$$

Parallel sweep rows are separated by row spacing Δy.

VI. Search Route Optimization

When operators enter multiple search locations, search order directly impacts total flight distance and mission duration. MANAR incorporates a mathematically independent greedy nearest-neighbor route optimizer.

A. Problem Formulation

Let the operator define a set of n search locations:

$$L = \{L_1, L_2, \ldots, L_n\}, \qquad L_i = (\phi_i, \lambda_i) \tag{13}$$

Let $p_0 = (\phi_{\text{drone}}, \lambda_{\text{drone}})$ denote the aircraft's position at plan lock time, and let $h = (\phi_{\text{home}}, \lambda_{\text{home}})$ denote home base coordinates.

Evaluating exact Traveling Salesman Problem (TSP) solutions via naive exhaustive permutation enumeration requires evaluating all $n!$ waypoint orderings [4], which grows factorially and is inefficient for embedded execution.

B. Greedy Nearest-Neighbor Algorithm

MANAR implements an $O(n^2)$ greedy nearest-neighbor algorithm in `route_optimizer.cpp`. Starting from initial position p_0, the algorithm iteratively selects the closest unvisited search location.

Let U_k denote the set of unvisited locations at step k, with $U_0 = L$. For $k = 0, 1, \ldots, n-1$:

$$p_{k+1} = \underset{x \in U_k}{\operatorname{argmin}}\ d(p_k, x) \tag{14}$$

The selected location p_{k+1} is appended to the ordered route sequence, and the unvisited set is updated:

$$U_{k+1} = U_k \setminus \{p_{k+1}\} \tag{15}$$

Home location h is excluded from greedy candidate selection iterations. The final search-to-home leg distance $d(p_n, h)$ is included when calculating total planned transit distance d_{total}, but home is not inserted as an entry into the ordered search location list:

$$d_{\text{total}} = d(p_0, p_1) + \sum_{k=1}^{n-1} d(p_k, p_{k+1}) + d(p_n, h) \tag{16}$$

C. Combinatorial Analysis

The computational complexity of the greedy route optimizer is $O(n^2)$. At step k, evaluating distance to remaining candidates requires $|U_k| = n - k$ Haversine calculations. The total number of distance evaluations N_{eval} is:

$$N_{\text{eval}} = \sum_{k=0}^{n-1} (n - k) = \frac{n(n+1)}{2} = O(n^2) \tag{17}$$

For $n = 8$ search locations, nearest-neighbor selection performs 36 candidate distance evaluations ($8+7+6+5+4+3+2+1 = 36$), whereas naive exhaustive permutation enumeration considers $8! = 40\,320$ possible waypoint orderings. This reduces route-ordering work to quadratic candidate-distance evaluations and avoids factorial route enumeration.

D. Operator Override Privilege

In tactical SAR operations, human operators may possess domain knowledge (e.g., wind drift estimates, high-probability intelligence, terrain barriers) that renders geometric nearest-neighbor routes suboptimal. MANAR enforces operator override privilege:

$$\text{Route} = \begin{cases} \text{GreedyOpt}(p_0, L, h), & \text{if operator selects } Y \\ L, & \text{if operator selects } N \end{cases} \tag{18}$$

When optimization is disabled (N), the input ordering entered by the operator is preserved identically ($p_k = L_k$). In both modes, execution IDs $(1, 2, \ldots, n)$ are assigned to the final sequence, while original entry indices are preserved under `input_id`.

E. Search Plan Locking

Once submitted, `control.exe` freezes the search plan (`searchplanlocked = true`). In the current prototype, a locked search plan remains immutable until the prototype reset path clears the lock.

VII. Multisensor Payload Architecture

MANAR incorporates a software model representing eleven controllable payload categories for victim search, identification, and signaling.

A. Payload Categories

The prototype models eleven controllable component categories in `components.cpp`. The descriptions below indicate their intended subsystem roles; sensing and processing functionality is not yet implemented:

1) **Thermal LWIR Camera:** Body heat detection.
2) **Day RGB Camera:** Visual inspection and terrain validation.
3) **Low-Light / IR Camera:** Low ambient light sensitivity.
4) **FMCW Radar:** Range, velocity, and micro-motion detection.
5) **Passive RF Sensor:** Mobile signal uplink detection.
6) **Public Address Emitter:** Audible instruction broadcast.
7) **Microphone Array:** Acoustic monitoring for vocal signals.
8) **Amber Beacon:** Visual alerting for ground teams.
9) **White Guidance Strobe:** Visual orientation strobe.
10) **Downward Spotlight:** Ground illumination.
11) **Smoke Marker Canister:** Visual smoke signaling.

B. Candidate Hardware Mapping

Table II maps software payload categories to candidate hardware under evaluation. Rows marked TBD indicate categories where hardware selection has not been evaluated.

C. Perception Pipeline Integration

In the current prototype, payload component states can be commanded individually via `CHANGE_COMPONENTS` or toggled automatically during mission hold states. Automated sensor fusion algorithms and ML perception models are reserved for future work.

TABLE II
Candidate Payload Hardware (Non-Final)

Category	Candidate Hardware
Thermal LWIR	FLIR Boson+ 640
Day RGB	Sony FCB-EV9500L
Low-Light IR	SiOnyx Aurora
FMCW Radar	TBD (band not finalized)
Passive RF	TBD
PA Emitter	TBD
Microphone Array	ReSpeaker v2.0
Visual Alerting	TBD
Smoke Marker	TBD

VIII. Safety and Software Failsafes

Safety rules and monitoring logic are enforced directly within the C++ control core.

A. Supervised Operator Control

Mission-critical state modifications—such as starting search missions (`START_MISSION`), initiating RTL (`RTL`), or launching the aircraft (`LAUNCH_DRONE`)—are operator initiated. Autonomous control logic enforces safety-oriented responses, such as battery failsafe triggers.

B. Battery Failsafe Hierarchy

The control core monitors simulated battery percentage B_k on every loop iteration:

$$\text{Action}(B) = \begin{cases} \text{Emergency Land} & B_k \le B_{\text{land}} \\ \text{Emergency RTL} & B_k \le B_{\text{emergency\_rtl}} \\ \text{Recommend RTL} & B_k \le B_{\text{rtl\_warn}} \\ \text{Warning} & B_k \le B_{\text{warn}} \\ \text{Normal Operation} & \text{otherwise} \end{cases} \tag{19}$$

When $B_k \le B_{\text{emergency\_rtl}}$, `activateRTL()` is invoked automatically to command Return-to-Launch navigation to home base. When $B_k \le B_{\text{land}}$, the control core executes the prototype stop-flight routine, records aircraft coordinates, and sets flight mode to stopped.

C. Software Command Clamps

All incoming velocity and altitude command arguments are validated against software safety clamps prior to execution:

$$v_{\text{cmd}} \le v_{\max} \tag{20}$$

$$H_{\text{cmd}} \le H_{\max} \tag{21}$$

Commands exceeding these bounds are clamped or rejected, logging `WARN` event entries to `runtime/logs.txt`.

IX. Software Architecture

MANAR is implemented as a modular C++ system structured into system domain modules and standalone application executables.

A. C++ Core Submodules

The core repository (`core/`) comprises:

- `apps/control.cpp`: Control loop, command parser (`checkcommands`), runtime serialization (`saveRuntime`), and command dispatch.
- `apps/terminal.cpp`: Operator terminal interface supporting coordinate entry, DMS parsing, optimization toggling, and manual commands.
- `apps/setup.cpp`: Utility for selecting active configuration slots and parameter templates.
- `system/mission.cpp`: Implements `mission` class, lawnmower sweep progression, search plan locking, and `activateRTL()`.
- `system/drone.cpp`: Implements `drone` and `home` classes, distance logic, and battery simulation rules.
- `system/flight.cpp`: Implements `flight` class, altitude/velocity clamps, and mode handling.
- `system/components.cpp`: Manages states for 11 controllable payload categories.
- `system/route_optimizer.cpp`: Haversine distance calculator and greedy nearest-neighbor route optimizer.
- `system/shared.cpp`: Timestamped structured plain-text event logger (`logEvent`) writing to `runtime/logs.txt`.

B. Separation of Software Layers

MANAR's architecture clearly separates the current functional C++ prototype from planned future interface and perception layers:

- **C++ Control Core (`control.exe`):** Currently operational. Executes control logic, route planning, and state management.
- **Operator Interface (React/TypeScript):** Selected target GUI framework. Will communicate asynchronously with the C++ control core via WebSocket + JSON IPC protocol.
- **Perception Pipeline (Python/PyTorch):** Reserved for future ML target detection and multisensor evidence fusion development.

X. Functional Prototype Verification

The prototype implementation of MANAR has been built and functionally verified under Windows 11 using GCC 16.1.0 (`g++`).

A. Build System

Core applications are compiled into standalone executables:

```
g++ apps/control.cpp system/shared.cpp
  system/flight.cpp system/components.cpp
  system/drone.cpp system/mission.cpp
  system/route_optimizer.cpp
  -I. -Ithird_party -o build/control.exe
g++ apps/terminal.cpp -I. -Ithird_party
  -o build/terminal.exe
g++ apps/setup.cpp -I. -Ithird_party
  -o build/setup.exe
```

Listing 1. MANAR compilation commands.

TABLE III
Prototype Verification Test Parameters

Symbol	Parameter	Value
v_Q	Quick velocity	15.0 m/s
v_A	Active velocity	8.0 m/s
v_I	Inspect velocity	2.0 m/s
v_H	Hover velocity	0.0 m/s
H_{launch}	Launch altitude	10.0 m
$H_{\max}$	Altitude ceiling	2000 m
$v_{\max}$	Speed ceiling	15.0 m/s
$W \times H$	Sector dimensions	100×100 m
Δy	Row spacing	20.0 m
R_{reach}	Waypoint arrival radius	5.0 m
R_{home}	Home arrival radius	100 m
B_{warn}	Warning threshold	30%
$B_{\text{rtl\_w}}$	Recommend RTL	25%
B_{emg}	Emergency RTL	15%
B_{land}	Emergency Land	5%
α_m	Battery sim. coefficients	$5, 3, 2, 1$

B. Functional Verification Results

Functional testing confirmed the following behaviors:

1) **Route Optimization vs. Manual Order:** For three search coordinates (L_1, L_2, L_3) entered in non-optimal geometric order relative to start position p_0, enabling optimization (Y) produced a shorter planned transit distance than the entered manual ordering ($d_{\text{greedy}} < d_{\text{manual}}$), while disabling optimization (N) preserved exact entry order.
2) **Search Plan Locking:** Submitting locations locked `search_locations.json`. Subsequent `SET_SEARCH_LOCATIONS` commands were rejected with `WARN` logs until the prototype reset path cleared the lock.
3) **Start Rejection on Empty Plan:** Attempting `START_MISSION` without search locations resulted in immediate rejection by both terminal and control core.
4) **RTL Precedence:** Triggering `RTL` during multi-sector search cleared search waypoints, reset `search_locations.json` to a single home destination, launched the aircraft if grounded, and initiated return navigation using the configured flight mode.

C. Prototype Test Configuration

Table III lists the configuration parameters used during functional verification. These values are implementation-specific test inputs and do not constitute finalized MANAR operational limits.

XI. Limitations and Future Work

MANAR's current implementation establishes a deterministic C++ baseline, but several limitations warrant future development:

- **Heuristic Optimality Bounds:** The greedy nearest-neighbor algorithm provides efficient $O(n^2)$ execution but does not guarantee global route optimality. Local

improvement heuristics (e.g., 2-opt) or exact MILP optimization can be investigated where problem size permits.

- **Software Battery Simulation:** The current prototype uses a discrete mode-dependent battery simulation rule to exercise failsafe logic. Future revisions will integrate realistic electro-aerodynamic battery discharge models based on voltage, current, and wind drag.
- **Airspace and Wind Dynamics:** Velocity calculations assume still air. Incorporating wind vector estimation and dynamic 3D geofencing will improve navigation accuracy in real-world environments.

Future work will focus on implementing the React/TypeScript GUI, developing Python/PyTorch perception models, integrating PX4 Software-in-the-Loop (SITL) simulation, and conducting field deployment validation.

XII. Conclusion

This paper presented MANAR (Multi-sensor Autonomous Navigation and Aerial Rescue), a supervised-autonomy SAR system architecture built upon a deterministic C++ control core, Haversine geographic navigation, and an $O(n^2)$ greedy search route optimizer with operator override privilege. By maintaining a strict distinction between the implemented functional C++ prototype, selected software state ownership architecture, and planned perception extensions, the paper establishes a grounded technical baseline. Functional prototype verification confirmed deterministic control behavior, search-plan locking, automatic Return-to-Launch (RTL) navigation preemption, and timestamped structured plain-text event logging. MANAR establishes a foundation for dependable, human-supervised aerial search-and-rescue robotics.

References

[1] M. Silvagni, A. Tonoli, M. Zenerole, and M. Chiaberge, "Multipurpose uav for search and rescue operations in mountain avalanche events," *Geomatics, Natural Hazards and Risk*, vol. 8, no. 1, pp. 18–33, 2017.

[2] M. A. Goodrich, B. S. Morse, D. Gerhardt, J. L. Cooper, M. Quigley, J. A. Adams, and C. Humphrey, "Towards human-robot teams in wilderness search and rescue," *Journal of Field Robotics*, vol. 25, no. 1-2, pp. 69–89, 2008.

[3] R. W. Sinnott, *Virtues of the Haversine*, 1984, vol. 68, no. 2.

[4] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, "The traveling salesman problem: A computational study," 2006.